



Licenciatura em Telecomunicações e Informática

Arquitetura e Tecnologia de Computadores temporizadores

Paulo Cardoso

Grupo de Sistemas Embebidos
Departamento de Eletrónica Industrial
Escola de Engenharia
Universidade do Minho





Sumário



Embedded Systems
Research Group

- Definições
- Rotinas de atraso
- Temporizadores
- Configuração
- Operação



- Registos, o que são?
 - São células de memória
 - Com uma funcionalidade específica e não usadas como memória “normal”
 - São usados dentro do processador
 - Para guardar “valores intermédios” nos cálculos
 - Nos microcontroladores são também usados fora do processador
 - Associados aos periféricos
 - Registos de configuração
 - Registos de dados



- No 8051
 - O registros que lidam com os periféricos são designados SFRs (Special Function Registers)
 - Cada periférico contém
 - Registos de configuração
 - Escreve-se um valor de forma a definir um comportamento para o periférico
 - Registos de dados
 - Contêm os dados a enviar/receber do periférico
 - Nem todos os periféricos manipulam dados



• Registos, o que são? - Exemplo

SFR Definition 19.1. CLKSEL: Clock Select

Bit	7	6	5	4	3	2	1	0
Name		Reserved			OUTCLK	CLKSL[2:0]		
Type	R	R/W			R/W	R/W		
Reset	0	0	0	0	0	0	0	0

SFR Address = 0xA9; SFR Page = All Pages

Bit	Name	Function
7	Unused	Read = 0b; Write = don't care
6:4	Reserved	Read = 0b; Must Write 000b.
3	OUTCLK	Crossbar Clock Out Select. If the SYCLK signal is enabled on the Crossbar, this bit selects between outputting SYCLK and SYCLK synchronized with the Port I/O pins. 0: Enabling the Crossbar $\overline{\text{SYCLK}}$ signal outputs SYCLK. 1: Enabling the Crossbar $\overline{\text{SYCLK}}$ signal outputs SYCLK synchronized with the Port I/O.
2:0	CLKSL[2:0]	System Clock Source Select Bits. 000: SYCLK derived from the Internal High-Frequency Oscillator / 4 and scaled per the IFCN bits in register OSCICN. 001: SYCLK derived from the External Oscillator circuit. 010: SYCLK derived from the Internal High-Frequency Oscillator / 2. 011: SYCLK derived from the Internal High-Frequency Oscillator. 100: SYCLK derived from the Internal Low-Frequency Oscillator and scaled per the OSCLD bits in register OSCLCN. 101-111: Reserved.



- Registos, o que são? - Exemplo
 - O registo chama-se **CLKSEL** (*datasheet*: pag 138)

SFR Definition 19.1. CLKSEL: Clock Select

Bit	7	6	5	4	3	2	1	0
Name		Reserved			OUTCLK	CLKSL[2:0]		
Type	R	R/W			R/W	R/W		
Reset	0	0	0	0	0	0	0	0

SFR Address = 0xA9; SFR Page = All Pages

- Trata-se de um registo de 8 bits
- Tem dois campos **CLKSL** e **OUTCLK**
 - Ambos de leitura e escrita
- A configuração por defeito é **0x00**
- A sua localização na memória é no endereço **0xA9**



- Registos, o que são? - Exemplo

Bit	Name	Function
7	Unused	Read = 0b; Write = don't care
6:4	Reserved	Read = 0b; Must Write 000b.
3	OUTCLK	Crossbar Clock Out Select. If the $\overline{\text{SYSCLK}}$ signal is enabled on the Crossbar, this bit selects between outputting $\overline{\text{SYSCLK}}$ and $\overline{\text{SYSCLK}}$ synchronized with the Port I/O pins. 0: Enabling the Crossbar $\overline{\text{SYSCLK}}$ signal outputs $\overline{\text{SYSCLK}}$. 1: Enabling the Crossbar $\overline{\text{SYSCLK}}$ signal outputs $\overline{\text{SYSCLK}}$ synchronized with the Port I/O.
2:0	CLKSL[2:0]	System Clock Source Select Bits. 000: $\overline{\text{SYSCLK}}$ derived from the Internal High-Frequency Oscillator / 4 and scaled per the IFCN bits in register OSCICN. 001: $\overline{\text{SYSCLK}}$ derived from the External Oscillator circuit. 010: $\overline{\text{SYSCLK}}$ derived from the Internal High-Frequency Oscillator / 2. 011: $\overline{\text{SYSCLK}}$ derived from the Internal High-Frequency Oscillator. 100: $\overline{\text{SYSCLK}}$ derived from the Internal Low-Frequency Oscillator and scaled per the OSCLD bits in register OSCLCN. 101-111: Reserved.

- Tem dois campos: **CLKSL** e **OUTCLK**
 - Do bit 0 ao bit 2 é definida a configuração do campo **CLKSL**
 - Como são 3 bits a configuração pode ser 000, 001...111
 - O datasheet diz o que cada configuração representa
 - No bit 3 é definida a configuração do **OUTCLK**



- Registos, o que são? - Exemplo
 - Como se acede ao registo?
 - Como se fosse uma variável
 - Ex: **CLKSEL = 0x02**
 - **0x02 = 0000.0010**, logo **OUTCLK=0** e **CLKSL=010**

3	OUTCLK	Crossbar Clock Out Select. If the $\overline{\text{SYSCLK}}$ signal is enabled on the Crossbar, this bit selects between outputting $\overline{\text{SYSCLK}}$ and $\overline{\text{SYSCLK}}$ synchronized with the Port I/O pins. <u>0: Enabling the Crossbar $\overline{\text{SYSCLK}}$ signal outputs $\overline{\text{SYSCLK}}$.</u> 1: Enabling the Crossbar $\overline{\text{SYSCLK}}$ signal outputs $\overline{\text{SYSCLK}}$ synchronized with the Port I/O.
2:0	CLKSL[2:0]	System Clock Source Select Bits. 000: $\overline{\text{SYSCLK}}$ derived from the Internal High-Frequency Oscillator / 4 and scaled per the IFCN bits in register OSCICN. 001: $\overline{\text{SYSCLK}}$ derived from the External Oscillator circuit. <u>010: $\overline{\text{SYSCLK}}$ derived from the Internal High-Frequency Oscillator / 2.</u> 011: $\overline{\text{SYSCLK}}$ derived from the Internal High-Frequency Oscillator. 100: $\overline{\text{SYSCLK}}$ derived from the Internal Low-Frequency Oscillator and scaled per the OSCLD bits in register OSCLCN. 101-111: Reserved.



Temporizadores

introdução

- No controlo de processos
 - Muitas vezes é necessário
 - Contar tempo
 - Incrementalmente
 - Contar eventos
 - Ex. quantos produtos passaram no tapete rolante



- O que tem de mal uma rotina de *delay*?

- Exemplo

```
void delay(int value) {  
    int x;  
  
    for (x=0;x<value;x++);  
  
}
```

- Na invocação

```
delay(100);
```

- Quanto tempo de atraso vou gerar?
 - 100 s?
 - 100 ms?
 - Um tempo...



- O que tem de mal uma rotina de *delay*?
 - Esta rotina está baseada na execução de instruções C
 - Que são traduzidas para código assembly/máquina
 - Que são executadas pelo processador



- A execução do código depende de
 - Da arquitetura do microcontrolador
 - As arquiteturas pipelined (como o nosso micro)
 - Paralelizam a execução das instruções
 - O que complica as contas das instruções....
 - Da frequência do *clock* do microcontrolador
 - Pois determina o ciclo de instrução (fetch decode execute)
 - OK, até podemos controlar
 - Se souber a freq. do *clock* e com contas
 - Na verdade os *timers* também podem ter esta dependência



- A execução do código depende de (cont.)
 - Do compilador
 - Que pode traduzir este código C de forma diferente
 - OK, até podemos controlar,
 - Vendo o código gerado e fazendo contas
 - Mais complicado numa arquitetura *pipelined*
- Da aplicação
 - Se houver uma interrupção: a execução da rotina para!
 - NÃO PODEMOS CONTROLAR!!!
 - Enquanto se conta tempo
 - O micro não faz outra coisa...



- A solução para a contagem de tempo
 - *Hardware* dedicado
 - Um recurso existente em qualquer microcontrolador
 - Designados por temporizadores ou *timers*
- Temporizadores do C8051F388
 - Seis temporizadores/contadores
 - Dois são compatíveis com o 8051 original: Timer0, Timer1
 - *Timers/counters* de 16 bits
 - Vamos apenas dedicar atenção a estes!



- A utilização dos *timers* (ou qualquer outro periférico)
 - Está dividida em duas partes
 - Configuração
 - Geralmente os periféricos têm vários modos de operação
 - Parametrizados através de registos de configuração (SFRs)
 - Operação
 - Depois de configurado o periférico está pronto a ser usado
 - Os dados sobre os quais os periféricos operam estão também em registos (SFRs)
 - Outros registos ou *flags* (bits) poderão ser alterados em função do estado dos dados (SFRs)



- Configuração
 - Para usar os *timers* temos de configurar
 - 1) O relógio do sistema
 - Para definir o *clock* do sistema
 - 2) O relógio dos *timers*
 - Para definir os *clocks* dos *timers*
 - 3) Modo de funcionamento
 - Pode funcionar como *timer* ascendente/descendente ou contador
 - Aqui é onde se faz a configuração do *timer* propriamente dita



- Configuração
 - 1) O relógio do sistema
 - 2) O relógio dos *timers*
 - 3) Modo de funcionamento



- 1) O relógio do sistema
 - "Define" o *clock* para todo *hardware*
 - Quando o micro arranca
 - Por defeito usa um oscilador interno de alta frequência
 - Calibrado para 48MHz por defeito
 - De acordo com o registo **CLKSEL**
 - Por defeito o oscilador é dividido por 4 → 12Mhz
 - E a frequência final é dada pelo registo **OSCICN**
 - De acordo com o registo **OSCICN**
 - Por defeito esta frequência é dividida ainda por 8
 - Dando uma frequência final de 1,5MHz



- 1) O relógio do sistema
 - Registo **CLKSEL**

SFR Definition 19.1. CLKSEL: Clock Select

Bit	7	6	5	4	3	2	1	0
Name		Reserved			OUTCLK	CLKSL[2:0]		
Type	R	R/W			R/W	R/W		
Reset	0	0	0	0	0	0	0	0

SFR Address = 0xA9; SFR Page = All Pages

2:0	CLKSL[2:0]	System Clock Source Select Bits. 000: SYSCLK derived from the Internal High-Frequency Oscillator / 4 and scaled per the IFCN bits in register OSCICN. 001: SYSCLK derived from the External Oscillator circuit. 010: SYSCLK derived from the Internal High-Frequency Oscillator / 2. 011: SYSCLK derived from the Internal High-Frequency Oscillator. 100: SYSCLK derived from the Internal Low-Frequency Oscillator and scaled per the OSCLD bits in register OSCLCN. 101-111: Reserved.
-----	------------	---



- 1) O relógio do sistema
 - Registo **OSCICN**

SFR Definition 19.3. OSCICN: Internal H-F Oscillator Control

Bit	7	6	5	4	3	2	1	0
Name	IOSCN	IFRDY	SUSPEND				IFCN[1:0]	
Type	R/W	R	R/W	R	R	R	R/W	
Reset	1	1	0	0	0	0	0	0

SFR Address = 0xB2; SFR Page = All Pages

1:0	IFCN[1:0]	Internal H-F Oscillator Frequency Divider Control Bits. The Internal H-F Oscillator is divided by the IFCN bit setting after a divide-by-4 stage. <u>00: SYSCLK can be derived from Internal H-F Oscillator divided by 8 (1.5 MHz).</u> 01: SYSCLK can be derived from Internal H-F Oscillator divided by 4 (3 MHz). 10: SYSCLK can be derived from Internal H-F Oscillator divided by 2 (6 MHz). 11: SYSCLK can be derived from Internal H-F Oscillator divided by 1 (12 MHz).
-----	-----------	---



- 1) O relógio do sistema
 - Exemplo
 - O que significa o código abaixo?

```
#include <c8051f380.h>
//__sfr16 __at (0x8C8A) TMR0; /* registo contagem Timer 0*/

void main (void) {
    unsigned int reload = 65036; // Tem de contar 500 ciclos: 65536-500

    PCA0MD = 0x00; // Desliga watchdog
    XBR1    = 0x40; // Ativa crossbar

    Definicao do relógio do sistema
    CLKSEL |= 0x02;

    //...
```



- Configuração
 - 1) O relógio do sistema
 - 2) O relógio dos *timers*
 - 3) Modo de funcionamento



- 2) O relógio dos *timers*
 - Sabendo/estabelecendo o relógio do micro
 - Um *timer* precisa de uma fonte de *clock*
 - Registo **CKCON**
 - Cada *timer* pode ter uma fonte de *clock* diferente



- 2) O relógio dos *timers*
 - Registo **CKCON**

SFR Definition 25.1. CKCON: Clock Control

Bit	7	6	5	4	3	2	1	0
Name	T3MH	T3ML	T2MH	T2ML	T1M	T0M	SCA[1:0]	
Type	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Reset	0	0	0	0	0	0	0	0

3	T1	Timer 1 Clock Select. Selects the clock source supplied to Timer 1. Ignored when C/T1 is set to 1. 0: Timer 1 uses the clock defined by the prescale bits SCA[1:0]. 1: Timer 1 uses the system clock.
2	T0	Timer 0 Clock Select. Selects the clock source supplied to Timer 0. Ignored when C/T0 is set to 1. 0: Counter/Timer 0 uses the clock defined by the prescale bits SCA[1:0]. 1: Counter/Timer 0 uses the system clock.
1:0	SCA[1:0]	Timer 0/1 Prescale Bits. These bits control the Timer 0/1 Clock Prescaler: 00: System clock divided by 12 01: System clock divided by 4 10: System clock divided by 48 11: External clock divided by 8 (synchronized with the system clock)



- 2) O relógio dos *timers*
 - Um *timer* precisa de uma fonte de *clock*
 - Registo **CKCON**
 - O campo **T0M**, **T1M** respetivamente dos timers 0 e 1
 - Dizem se os timers
 - são alimentados pelo relógio do sistema (1,5MHz por defeito) ou
 - se o seu clock ainda é subdividido (*prescaler*)
 - O campo **SCA0**, **SCA1** respetivamente dos timers 0 e 1
 - Define qual a divisão a que o relógio do sistema está sujeito se for essa a opção no campo acima



- 2) O relógio dos *timers*
 - Sabemos a frequência do micro
 - 1,5Mhz (por defeito), 48MhZ
 - Sabemos, analisando o esquemático e lendo o *datasheet*
 - Do *slide* anterior ficamos a saber
 - Se usar o *system clock* (**TxM=1**)
 - $f=1,5\text{MhZ} \rightarrow T = 0,6(6)\mu\text{s}$
 - Se subdividir o relógio do sistema (**TxM=0**)
 - **SCA=00** $\rightarrow f=1,5/12= 125 \text{ kHz} \rightarrow T = 8\mu\text{s}$ (valor por defeito)
 - **SCA=01** $\rightarrow f=1,5/4= 375 \text{ kHz} \rightarrow T = 2,6(6)\mu\text{s}$
 - **SCA=10** $\rightarrow f=1,5/48= 31,35 \text{ kHz} \rightarrow T = 32\mu\text{s}$
 - A cada período (T) dá-se um incremento na contagem



- 2) O relógio dos *timers*
 - Exemplo
 - O que significa o código abaixo?

```
#include <c8051f380.h>
//__sfr16 __at (0x8C8A) TMR0; /* registo contagem Timer 0*/

void main (void) {
    unsigned int reload = 65036; // Tem de contar 500 ciclos: 65536-500

    PCA0MD = 0x00; // Desliga watchdog
    XBR1    = 0x40; // Ativa crossbar

    //Definicao do relógio do sistema
    CLKSEL |= 0x02;

    //Definicao do relógio dos timers
    CKCON |= 0x02;
    //...
```



- 2) O relógio dos *timers*
 - Registo **CKCON**
 - Outros exemplos de configuração
 - `CKCON |= 0x0C /* CKCON |= 00001100 */`
 - *Timer 1 e 0* usam o *clock* do sistema
 - `CKCON |= 0x00 /* CKCON |= 00000000 */`
 - *Timer 1 e 0* usam o *clock* do sistema dividido por 12
 - `CKCON |= 0x09 /* CKCON |= 00001001 */`
 - *Timer 1* usa o *clock* do sistema
 - *Timer 0* usa o *clock* do sistema dividido por 4



- Configuração
 - 1) O relógio do sistema
 - 2) O relógio dos *timers*
 - 3) Modo de funcionamento



- 3) Modo de funcionamento
 - Counter
 - Conta eventos
 - Timer
 - Conta tempo
 - Modo 0: *timer* de 13 bits
 - Modo 1: *timer* de 16 bits
 - Modo 2: *timer* de 8 bits com *auto reload*
 - O registo de contagem é **TLx**. Quando transborda este é automaticamente carregado com o valor de **THx**
 - Modo 3: Dois *timers* de 8 bits (gerido pelo *timer* 0)



- 3) Modo de funcionamento
 - Princípio de funcionamento do *timer*
 - Existe um registo contador (**TL,TH**)
 - A cada ciclo do relógio (do timer e não do sistema)
 - (**TL,TH**) ++
 - Incrementa um ao registo
 - Quando o registo contador transborda
 - É ativada uma *flag* sinalizando o evento
 - A contagem recomeça automaticamente



- 3) Modo de funcionamento
 - Vários modos de contagem
 - Modo 0: *timer* de 13 bits
 - Modo 1: *timer* de 16 bits (Idêntico ao modo 0)
 - Modo 2: *timer* de 8 bits com *auto reload*
 - O registo de contagem é **TLx**. Quando transborda este é automaticamente carregado com o valor de **THx**
 - Modo 3: Dois *timers* de 8 bits (gerido pelo *timer* 0)



- 3) Modo de funcionamento
 - Configuração do *timer*
 - a) Selecionar contador vs temporizador (por defeito)
 - b) No temporizador
 - Selecionar o modo (modo 0 por defeito)
 - Definir valor inicial da contagem
 - Definir valor de recarga (*reload*)



- 3) Modo de funcionamento
 - Selecionar o modo: Registo **TMOD**

SFR Definition 25.4. TMOD: Timer Mode

Bit	Timer1				Timer0			
	7	6	5	4	3	2	1	0
Name	GATE1	C/T1	T1M[1:0]		GATE0	C/T0	T0M[1:0]	
Type	R/W	R/W	R/W		R/W	R/W	R/W	
Reset	0	0	0	0	0	0	0	0

SFR Address = 0x89; SFR Page = All Pages

- **GATE_x** **x=0 OU 1**
 - Se **GATE_x=1** o *timer/counter* só é ativado se o pino externo **INT1** estiver ativo
- **C/T_x**
 - **c=1** → Contador. Conta as transições descendentes do pino externo **T_x**
 - Serve por exemplo para contar entradas num estacionamento



- 3) Modo de funcionamento
 - Selecionar o modo: Registo **TMOD**

SFR Definition 25.4. TMOD: Timer Mode

Bit	Timer1				Timer0			
	7	6	5	4	3	2	1	0
Name	GATE1	C/T1	T1M[1:0]		GATE0	C/T0	T0M[1:0]	
Type	R/W	R/W	R/W		R/W	R/W	R/W	
Reset	0	0	0	0	0	0	0	0

SFR Address = 0x89; SFR Page = All Pages

- **T0M T1M**
 - Modo 0: *timer* de 13 bits
 - Existia no 8051 original por compatibilidade anterior (8048)
 - Manteve-se até hoje! Uma relíquia do passado...
 - Modo 1: *timer* de 16 bits
 - Modo 2: *timer* de 8 bits com *auto reload*
 - Modo 3: Dois *timers* de 8 bits (gerido pelo *timer* 0)



• 3) Modo de funcionamento

• Registo **TMOD**

• *Datasheet: pag 232*

SFR Definition 25.4. TMOD: Timer Mode

Bit	7	6	5	4	3	2	1	0
Name	GATE1	C/T1	T1M[1:0]		GATE0	C/T0	T0M[1:0]	
Type	R/W	R/W	R/W		R/W	R/W	R/W	
Reset	0	0	0	0	0	0	0	0

SFR Address = 0x89; SFR Page = All Pages

Bit	Name	Function
7	GATE1	Timer 1 Gate Control. 0: Timer 1 enabled when TR1 = 1 irrespective of $\overline{\text{INT1}}$ logic level. 1: Timer 1 enabled only when TR1 = 1 AND $\overline{\text{INT1}}$ is active as defined by bit IN1PL in register IT01CF (see SFR Definition 16.7).
6	C/T1	Counter/Timer 1 Select. 0: Timer: Timer 1 incremented by clock defined by T1M bit in register CKCON. 1: Counter: Timer 1 incremented by high-to-low transitions on external pin (T1).
5:4	T1M[1:0]	Timer 1 Mode Select. These bits select the Timer 1 operation mode. 00: Mode 0, 13-bit Counter/Timer 01: Mode 1, 16-bit Counter/Timer 10: Mode 2, 8-bit Counter/Timer with Auto-Reload 11: Mode 3, Timer 1 Inactive
3	GATE0	Timer 0 Gate Control. 0: Timer 0 enabled when TR0 = 1 irrespective of $\overline{\text{INT0}}$ logic level. 1: Timer 0 enabled only when TR0 = 1 AND $\overline{\text{INT0}}$ is active as defined by bit IN0PL in register IT01CF (see SFR Definition 16.7).
2	C/T0	Counter/Timer 0 Select. 0: Timer: Timer 0 incremented by clock defined by T0M bit in register CKCON. 1: Counter: Timer 0 incremented by high-to-low transitions on external pin (T0).
1:0	T0M[1:0]	Timer 0 Mode Select. These bits select the Timer 0 operation mode. 00: Mode 0, 13-bit Counter/Timer 01: Mode 1, 16-bit Counter/Timer 10: Mode 2, 8-bit Counter/Timer with Auto-Reload 11: Mode 3, Two 8-bit Counter/Timers



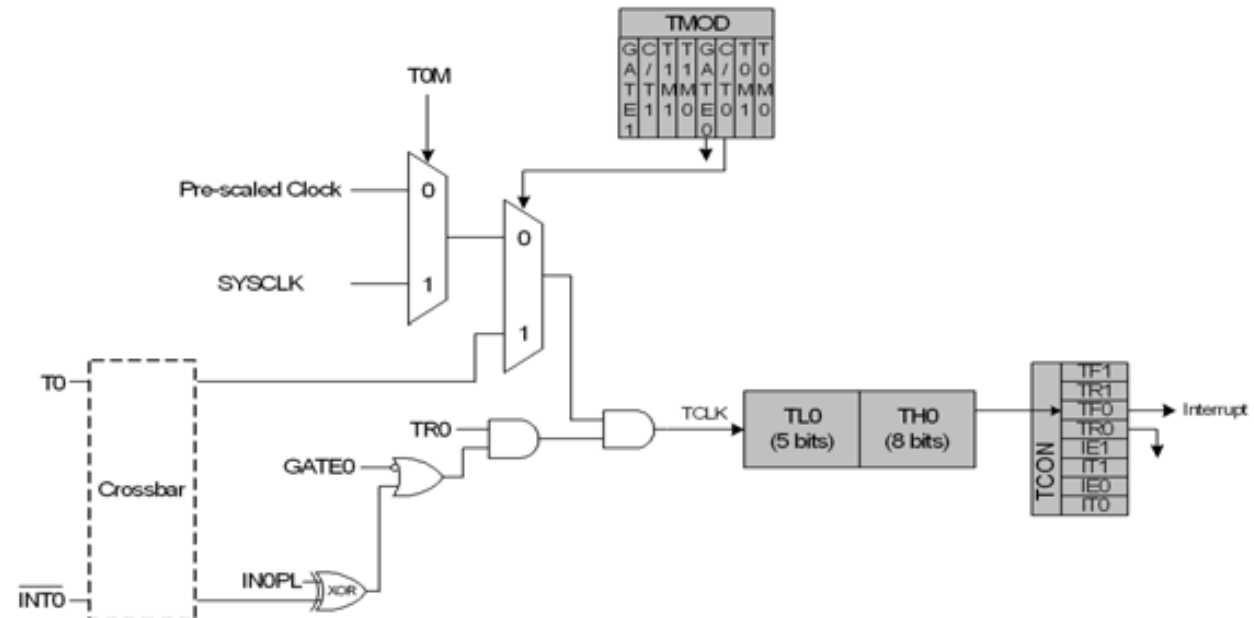
- 3) Modo de funcionamento
 - Valor da contagem: Registos **THx** e **TLx** (8 bits)
 - São os registos de contagem
 - Que devem ser “preenchidos” antes da contagem ser iniciada
 - Modo 0 (modo 13 bits)
 - **THx, TLx** formam um valor único
 - Apenas os 5 bits menos significativos de **THx** são considerados
 - Modo 1 (modo 16 bits)
 - **THx, TLx** formam um valor único
 - Os 16 bits são considerados
 - Modo 2 (modo 8 bits)
 - No registo **TLx** é colocado o valor da contagem
 - Geralmente igual a **THx**
 - No registo **THx** é colocado o valor de recarga (próximo slide)



- 3) Modo de funcionamento
 - Valor de recarga: Registro **TH_x** (no modo 2)
 - Modo 2 (modo 8 bits)
 - Quando se dá o transbordo no registo contador (**TL_x**)
 - Este não começa a contar do zero. Recebe o valor de **TH_x**
 - $TL_x \leftarrow TH_x$
 - Este modo pode ser simulado no modo 1
 - Através de código



- 3) Modo de funcionamento
 - Diagrama de blocos do *timer 0*
 - Exemplo: Configurado para o modo 0 em TMOD
 - O modo 1 é idêntico mas com contador de 16 bits





• 3) Modo de funcionamento

• Do *slide* anterior

- **TLO/THO** faz a contagem

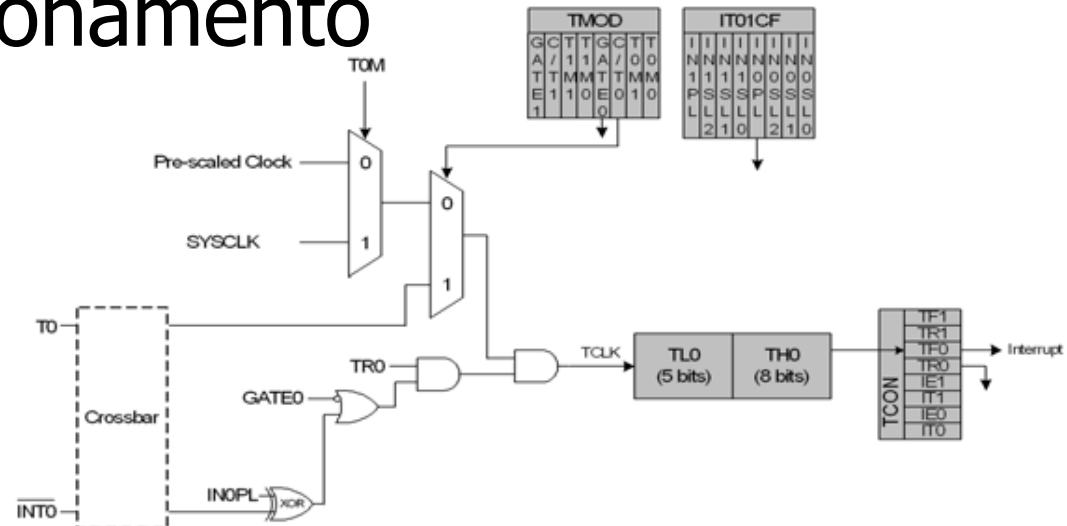
- Registo **TCON**

- **Flag TR0**

- Inicia a contagem

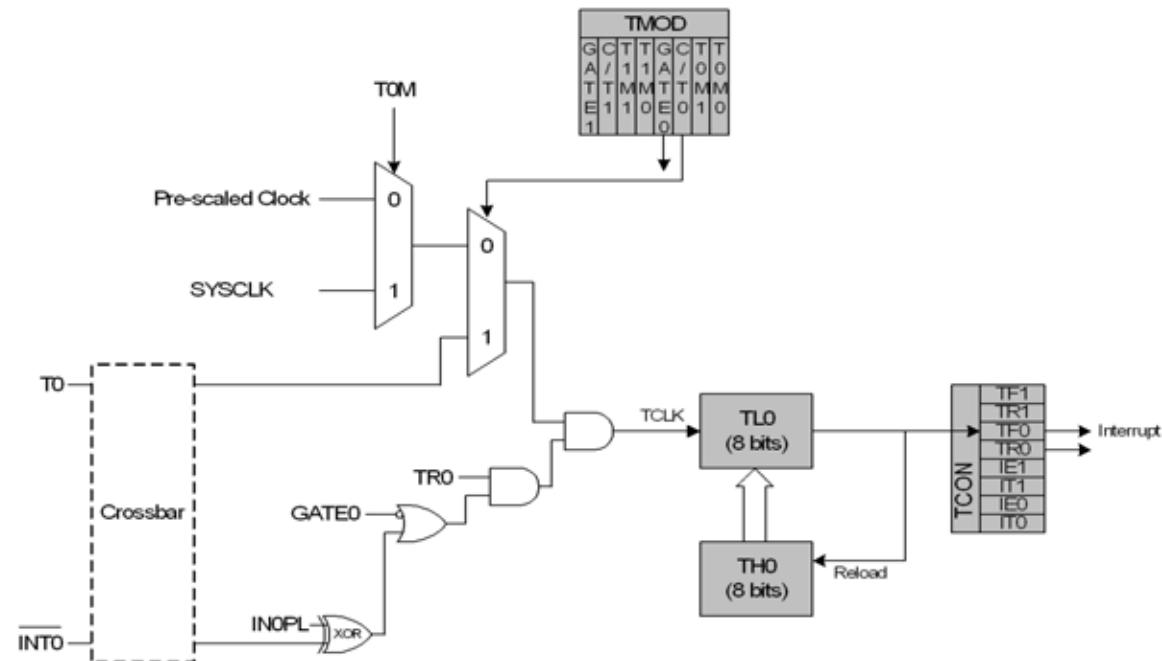
- **Flag TF0**

- Sinaliza o transbordo





- 3) Modo de funcionamento
 - Diagrama de blocos do *timer 0*
 - Exemplo: Configurado para o modo 2 em TMOD





• 3) Modo de funcionamento

• Do *slide* anterior

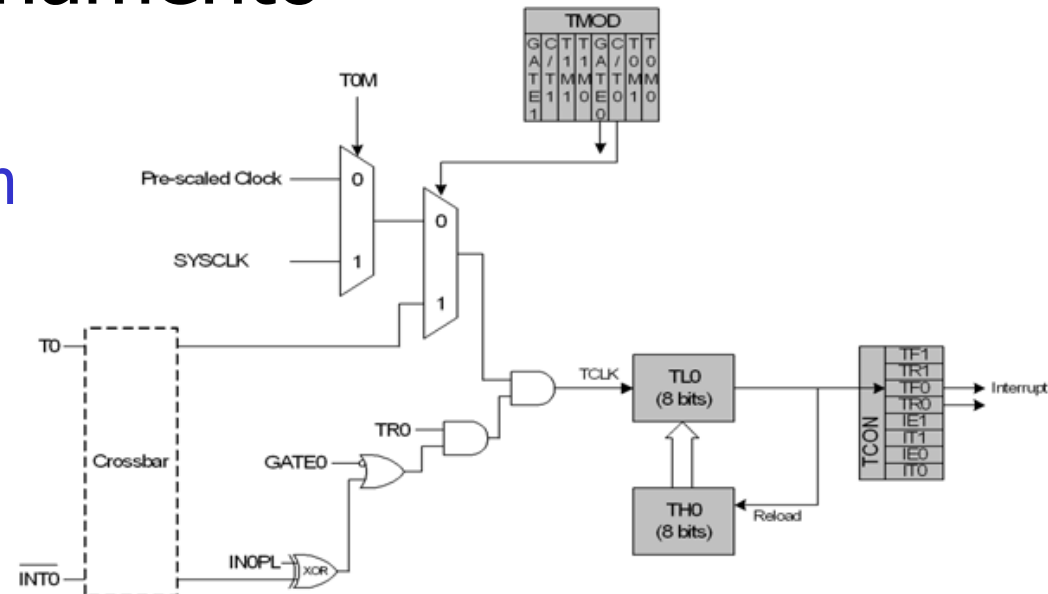
- **TL0** faz a contagem
- **TH0** recarrega o valor inicial de contagem
- Registo **TCON**

- **Flag TR0**

- Inicia a contagem

- **Flag TF0**

- Sinaliza o transbordo





- 3) Modo de funcionamento
 - Exemplo
 - O que significa o código abaixo?

```
#include <c8051f380.h>
//__sfr16 __at (0x8C8A) TMR0; /* registo contagem Timer 0*/

void main (void) {
    unsigned int reload = 65036; // Tem de contar 500 ciclos: 65536-500

    PCA0MD = 0x00; // Desliga watchdog
    XBR1    = 0x40; // Ativa crossbar

    //Definicao do relógio do sistema
    CLKSEL |= 0x02;

    //Definicao do relógio dos timers
    CKCON |= 0x02;

    //Modo de configuracao dos timers
    TMOD |= 0x01;
    //...
```



- Nota
 - Qual a diferença entre as atribuições abaixo?
 - `TMOD |= 0x01;`
 - `TMOD = 0x01;`



- Operação
 - Ativação da contagem
 - Detecção do fim da contagem
- Descrição geral do funcionamento
 - É colocado o valor a contar no registo de contagem (**TH0/TL0, TH1/TL1**). Já referido
 - É ativado o *timer* (**TR0, TR1**)
 - Quando se dá o transbordo de **ffff** para **0000**
 - É ativada uma *flag* (**TF0, TF1**)
 - No entanto o timer não para



- Ativação da contagem
 - Registo **TCON**
 - Bit **TR_x**

SFR Definition 25.3. TCON: Timer Control

Bit	7	6	5	4	3	2	1	0
Name	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
Type	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

SFR Address = 0x88; SFR Page = All Pages; Bit-Addressable



- Detecção de fim da contagem
 - Registo **TCON**
 - Bit **TF_x**
 - É ativado aquando do transbordo

SFR Definition 25.3. TCON: Timer Control

Bit	Timer1		Timer0					
	7	6	5	4	3	2	1	0
Name	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
Type	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

SFR Address = 0x88; SFR Page = All Pages; Bit-Addressable



- Problema 1
 - Criar uma onda quadrada de 500Hz em P2.0
 - Usando o micro a 24MHz
- Análise
 - $F = 500\text{Hz} \rightarrow T = 2\text{ms}$ (período da onda)
 - Como usar o *timer*?
 - Configura-lo de forma a que
 - A cada x ciclos faça o *toggle* em **P2.0**
 - Se o período da onda é 2ms, então
 - O tempo a alto é 1ms e o tempo a baixo é tb 1ms
 - Logo, $x = 1000\mu\text{s}$
 - Temos de criar um contador que conte 1000us



- Problema 1 - Análise
 - Fonte de relógio
 - Vamos alterar a frequência do micro
 - 24Mhz (**CLKSEL** |= 0x02)
 - Vamos configurar o *clock* do *timer*
 - Do registo **CKCON** ficamos a saber
 - Se usar o *system clock* (**TxM=1**)
 - $f=24\text{MHz} \rightarrow T = 0,041(6)\mu\text{s}$
 - Se usar o "desmultiplicador" (**TxM=0**)
 - **SCA=00** $\rightarrow f=24/12= 2\text{ MHz} \rightarrow T = 0,5\mu\text{s}$
 - **SCA=01** $\rightarrow f=24/4= 6\text{ MHz} \rightarrow T = 0,1(6)\mu\text{s}$
 - **SCA=10** $\rightarrow f=24/48= 0,5\text{ MHz} \rightarrow T = 2\mu\text{s}$
 - A cada período (T) dá-se um incremento na contagem



- Problema 1 - Análise
 - Assim, sabemos que
 - $SCA=10 \rightarrow T = 2\mu s$
(`CKCON |= 0x02`)
 - Temos então, ciclicamente
 - Contar $1000\mu s / 2\mu s = 500$ vezes e fazer o toggle de `P2.0`
 - Usando o *timer* no modo 2 (*autoreload*)
 - A cada transbordo carrega novamente o valor desejado
 - Só que o modo 2 usa um registo de 8 bits para *reload*
 - Só suporta valores 256 valores
 - Usando o *timer* no modo 1 (16 bits)
 - O *reload* é feito manualmente
 - Nota: estamos a usar apenas o Timer0 ou Timer1
 - O micro tem outros timers que suportam *autoreload* de 16 bits



- Problema 1 - Análise
 - Usando o *timer* no modo 2 (*auto-reload*)
 - Precisamos de contar até 500! (mas 8 bits → 256 max)
 - Alternativa: Contando 250 temos metade do valor
 - A cada dois transbordos temos 500 e fazemos o *toggle* da saída!

```
Enquanto (1)
    Enquanto (timer0 não transborda)
        [fica aqui]
    [flag transbordo] ← 0
    contador ++
    se contador == 2
        [complementar P2.0]
        contador ← 0
```

- De notar que o *timer* nunca para de contar
 - Em paralelo com a execução do código
- No modo 2 do timer o *auto-reload* é automático



- Problema 1 - Especificação – pseudo-código
 - Usando o *timer* no modo 2 (*auto-reload*)

```
CLKSEL ← 0x02
SCA ← modo2
Timer0 ← modo2
contador ← 0
reload ← 6
TH0, TL0 ← reload
[Iniciar timer0]
P2.0 ← 0
Enquanto(1)
    Enquanto (timer0 não transborda)
        [fica aqui]
        [flag transbordo] ← 0
        contador ++
        se contador == 2
            [complementar P2.0]
            contador ← 0
```

Porquê?



- Problema 1 – Implementação
 - Usando o *timer* no modo 2 (*auto-reload*)



Temporizadores exercícios



```
#include <c8051f380.h>
void main (void) {
    unsigned char reload = 6; // 256 - 250 = 6. Conta de 6 a FFFF
    unsigned char contador = 0;

    PCA0MD = 0x00; // Desliga watchdog
    XBR1    = 0x40; // Ativa crossbar

    CLKSEL |= 0x02; //SYSCLK derivado do Oscil/2=24MHz
    CKCON  |= 0x02; // Divide o clock por 48 => T=2us
    TMOD   |= 0x02; // Modo 8 bits Auto reload
    P2_0 = 0;      // Inicializa o valor do pino de saida

    TH0=reload;    // Inicializa o registo de contagem
    TL0=reload;
    TR0 = 1;       // Ativa a contagem
    while (1) {
        while (!TF0); // Aguarda fim da contagem
        TF0 = 0;     // Limpa a flag
        contador++;
        if (contador == 2) {
            contador = 0;
            P2_0 = !P2_0; // Complementa o pino do porto (saida)
        }
    }
}
```

Quando há transbordo
TL0←TH0 (automaticamente)
E a contagem continua



- Problema 1 - Implementação
 - Usando o *timer* no modo 2 (*auto-reload*)
 - Comprovar com o osciloscópio



- Problema 1 - Análise

- Usando o *timer* no modo 1 (*16 bits*)

- Precisamos de contar até 500!

- Sempre que há transbordo

- Complementamos o pino de saída

- Recarregamos o registo de contagem

```
Enquanto(1)
    Enquanto (timer0 não transborda)
        [fica aqui]
    TH0/TL0 ← reload
    [complementar P2.0]
```

- Embora o timer nunca pare de contar

- O *auto-reload* não é automático

- Há uma ligeira perturbação da contagem ao recarregar o registo



- Problema 1 - Especificação – pseudo-código
 - Usando o *timer* no modo 1 (*16 bits*)

```
CLKSEL ← 0x02
SCA ← modo2
Timer0 ← modo1
reload ← 65536-500

TH0/TL0 ← reload
[Iniciar timer0]
P2.0 ← 0
Enquanto(1)
    Enquanto (timer0 não transborda)
        [fica aqui]
    TH0/TL0 ← reload
    [complementar P2.0]
```



- Problema 1 – Implementação
 - Usando o *timer* no modo 1 (*16 bits*)



Temporizadores exercícios



```
#include <c8051f380.h>
void main (void) {
    unsigned int reload = 65036; // Tem de contar 500 ciclos: 65536-500

    PCA0MD = 0x00; // Desliga watchdog
    XBR1    = 0x40; // Ativa crossbar

    CLKSEL |= 0x02; //SYSCLK derivado do Oscil/2=24MHz
    CKCON  |= 0x02; // Divide o clock por 48 => T=2us
    TMOD   |= 0x01; // Modo 16 bits
    P2_0 = 0;      // Inicializa o valor do pino de saida

    TL0=reload; // Inicializa o registo de contagem
    TH0=reload >> 8;
    TR0 = 1;    // Ativa a contagem
    while (1) {
        while (!TF0); // Aguarda fim da contagem
        TL0=reload;   // Inicializa o registo de contagem
        TH0=reload >> 8;
        TF0 = 0;      //Limpa a flag
        P2_0 = !P2_0; // Complementa o pino do porto (saida)
    }
}
```

Qual o problema desta
implementação?



- Problema 1 - Implementação
 - Usando o *timer* no modo 1 (*16 bits*)
 - Comprovar com o osciloscópio



- Problema 2
 - Piscar um led a cada segundo em P2.0
 - Na verdade o led estará meio segundo ligado e meio segundo desligado
- Análise
 - Usando as configurações do problema 1
 - Timer 0 em *auto-reload*

$1s = 10^6us$. Assim terá de contar 500×10^3us (meio segundo para fazer o *toggle* da saída)

Contagens do timer

Se contarmos $250 \times 2us = 500us$

(como no problema anterior)

Se contarmos 1000 transbordos, temos o valor pretendido



- Problema 3
 - Criar uma onda quadrada de 25kHz em P2.0
 - Usar um *clock* e um modo diferente do *timer*



Temporizadores



Embedded Systems
Research Group



Obrigado